

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2613 - Inteligencia Artificial

Entrenamiento de Redes Neuronales

Hans Löbel

Dpto. Ingeniería de Transporte y Logística
Dpto. Ciencia de la Computación

Actualización de la Encuesta de Evaluación Docente

- La tasa aumentó!
- Pasó de 3,27% a 7,19% (de 5 a 11 respuestas)

Código	Profesor	Inscripciones	Respondido	Response Rate
IIC2613-1	Hans Albert Lobel Díaz	153	11	7,19%

En teoría, no hay diferencia entre la teoría y la práctica. En la práctica, sí la hay.

Tras bambalinas, para entrenar los modelos de DL adecuadamente se requiere una gran cantidad de detalles:

- ¿Cómo plantear y resolver el problema de aprendizaje de una red neuronal profunda?
- ¿Cómo controlar el sobreajuste en el entrenamiento?

Recordemos un poco cómo se ve un problema de aprendizaje en ML desde el punto de vista de la optimización

$$\operatorname{argmin}_W J(X, Y; W) = \lambda \mathcal{R}(W) + \sum_i^N \mathcal{L}(x_i, y_i; W)$$



Regularización



Suma de la pérdida por
cada ejemplo

Recordemos también algunos de los detalles formales de un **Perceptron**

- Si tenemos suficientes datos, pares (x_i, y_i) , es posible construir una función que nos indique cuán buena es en promedio la estimación del Perceptron.
- Sea $f(x; w) = \sigma(\sum_{i=1}^m w_{(i)} x_{(i)}) = \sigma(w \cdot x)$, una función que modela la aplicación de un Perceptrón parametrizado por un vector de pesos w , a un vector de características x .
- Sin suponer cosas raras sobre los datos, una posible **función de pérdida** (o costo, o error) puede quedar de la siguiente manera:

$$J(w) = \frac{1}{2n} \sum_{i=1}^n (f(x_i; w) - y_i)^2$$

- ¿Qué representa el valor mínimo de esta función, con respecto a los pesos w ?

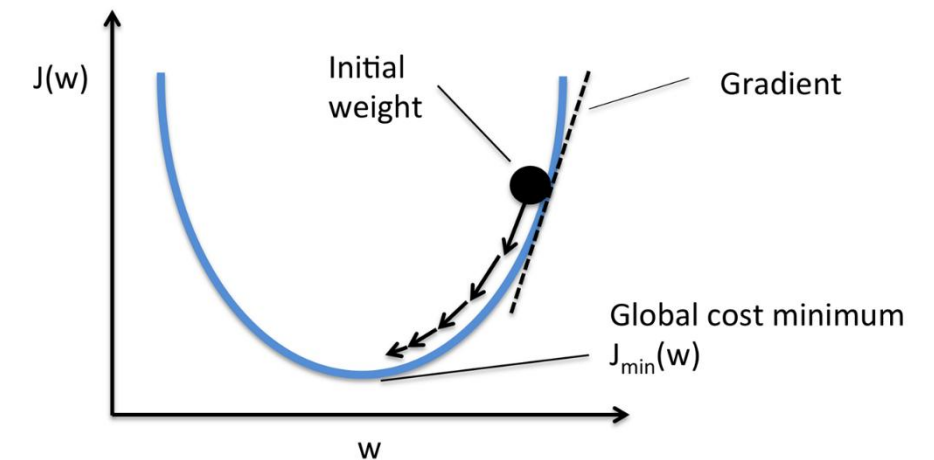
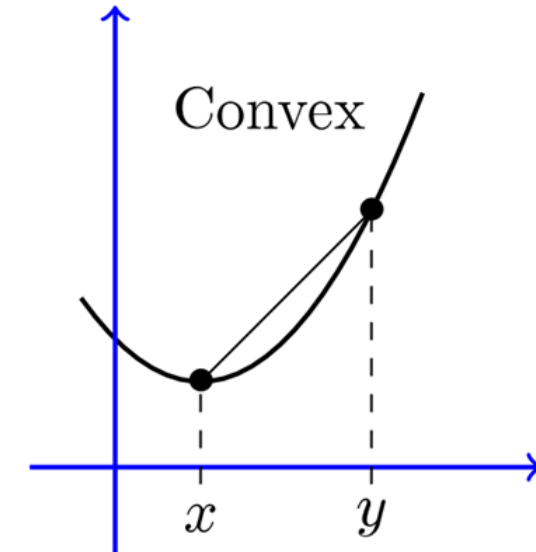
¿Cómo podemos entrenar un **Perceptron lineal** (i.e., con función identidad como función de activación) para estimar funciones?

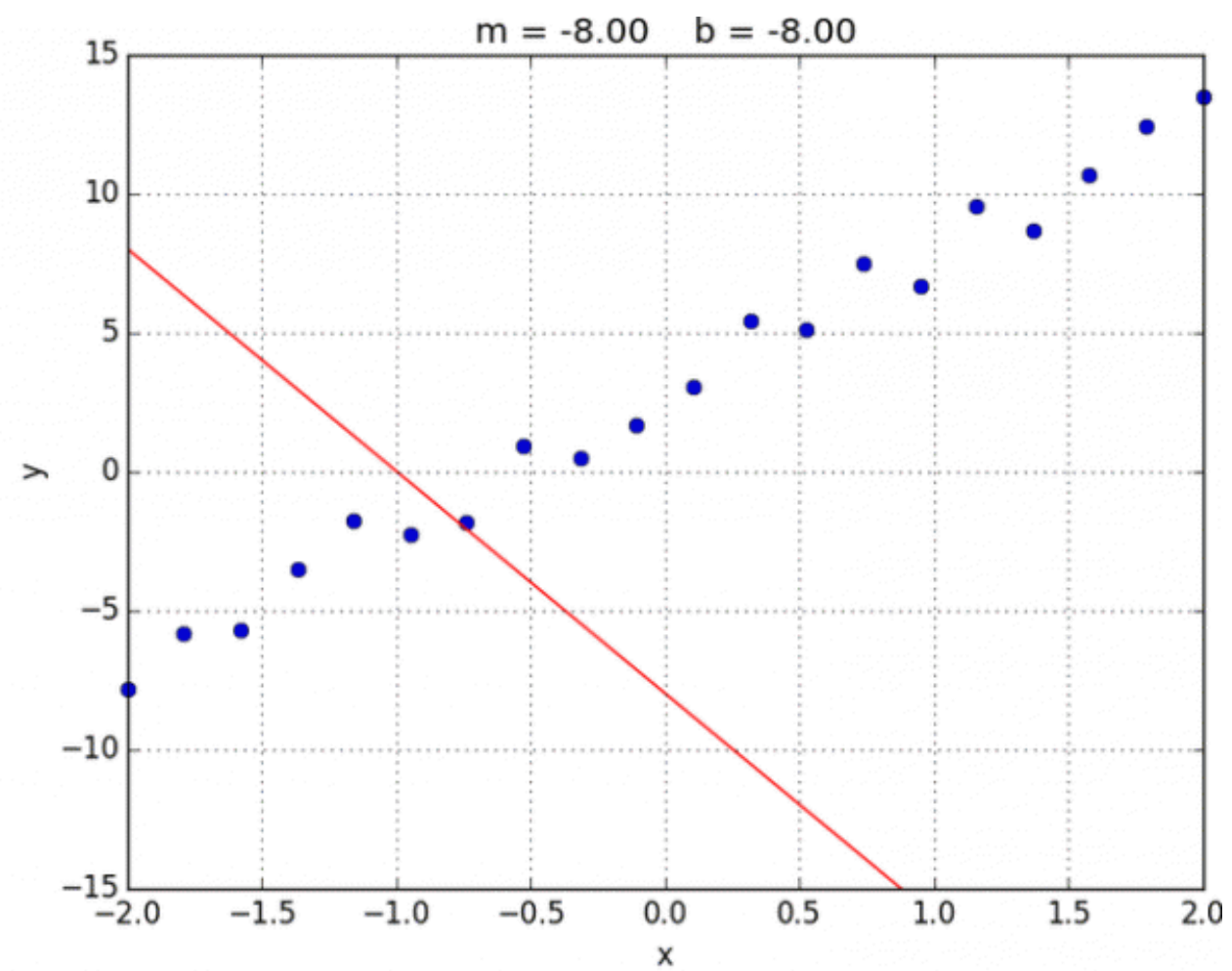
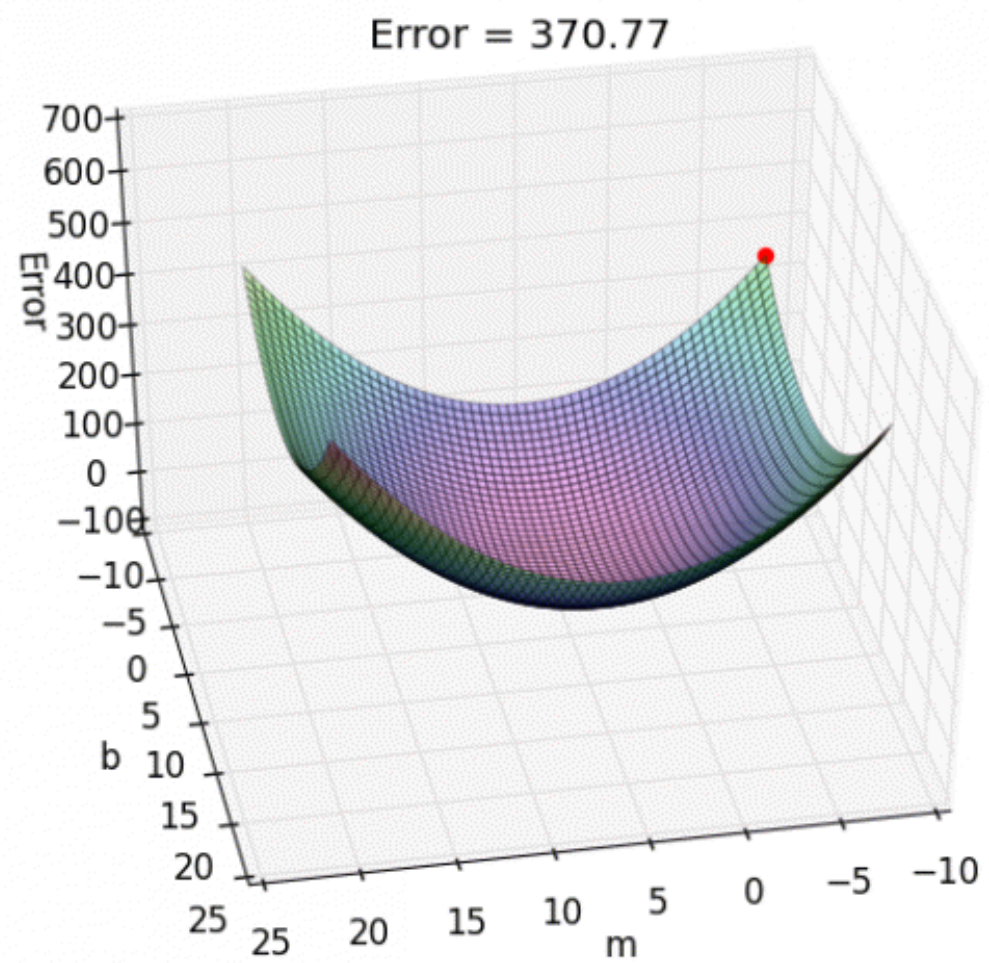
- La función de pérdida es convexa (siempre que la función de activación también lo sea, lo que ocurre en este y la mayoría de los casos)

$$J(w) = \frac{1}{2n} \sum_{i=1}^n (w \cdot x_i - y_i)^2$$

- Dado esto, es posible utilizar el algoritmo de descenso de gradiente

btw, ¿cuál es la derivada de la función de pérdida con respecto a w ?

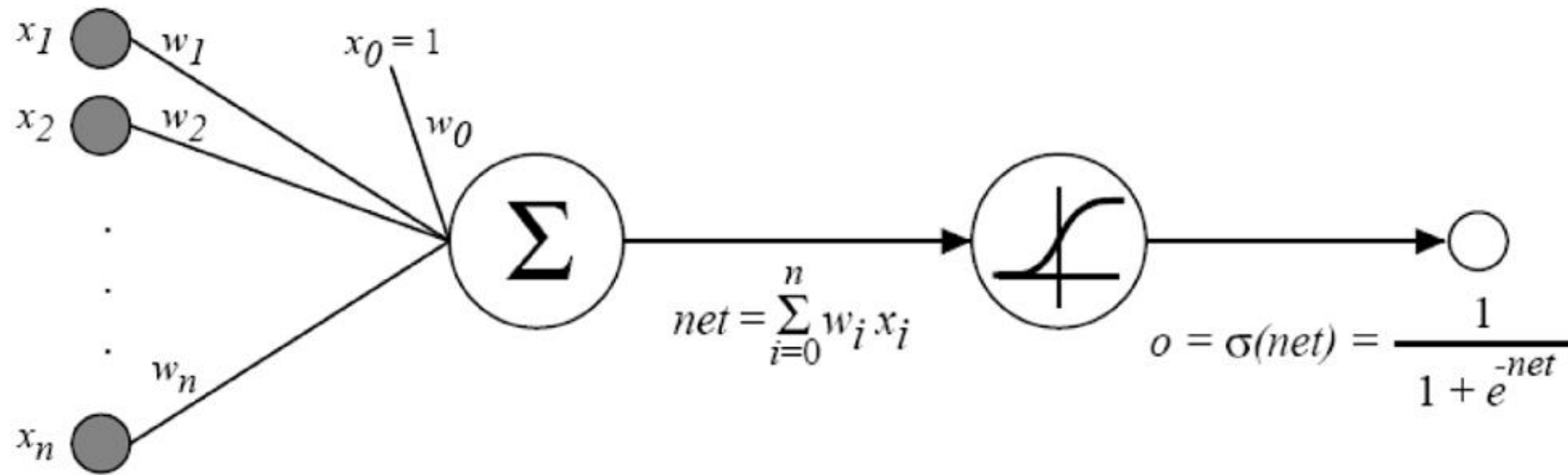




Dado que el problema de aprendizaje es convexo, podemos usar “tranquilamente” descenso de gradiente

Require: Pesos $w_{(i)}$ inicializados con un valor aleatorio pequeño
do
 $\Delta w \leftarrow \vec{0}$
 for each $(x, y) \in \mathcal{S}$ **do**
 Aplicar Perceptron a x y almacenar salida en $\hat{y}$
 for each $\Delta w_{(i)} \in \Delta w$ **do**
 $\Delta w_{(i)} \leftarrow \Delta w_{(i)} + (y - \hat{y})x_{(i)}$
 end for
 end for
 $w \leftarrow w - \alpha \frac{\Delta w}{|\mathcal{S}|}$
while condición de término == **False**

Si incorporamos una función de activación no lineal, la situación no cambia mucho



$$\frac{\partial J}{\partial w_i} = - \sum_{d \in D} (t_d - o_d) o_d (1 - o_d) x_{i,d}$$

Movámonos ahora a un MLP, que rápidamente expondrá las dificultades al entrenar redes neuronales

Siempre es bueno pensar también en las redes neuronales* como una composición de funciones:

- MLP de 1 capa (Perceptron):

$$f(x) = \sigma(W_1 x)$$

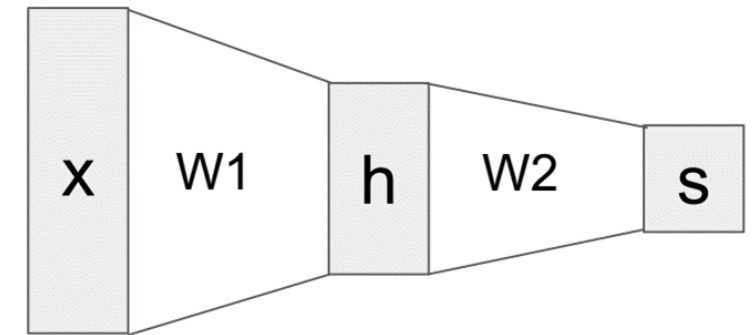
* $\sigma(z)$ = función de activación

- MLP de 2 capas:

$$f(x) = \sigma(W_2 \sigma(W_1 x))$$

- MLP de 3 capas:

$$f(x) = \sigma(W_3 \sigma(W_2 \sigma(W_1 x)))$$



* Con un poco de masaje algebraico, todo esto es también válido para una CNN, o cualquier otra red

¿Cómo queda entonces el problema de aprendizaje de un MLP?

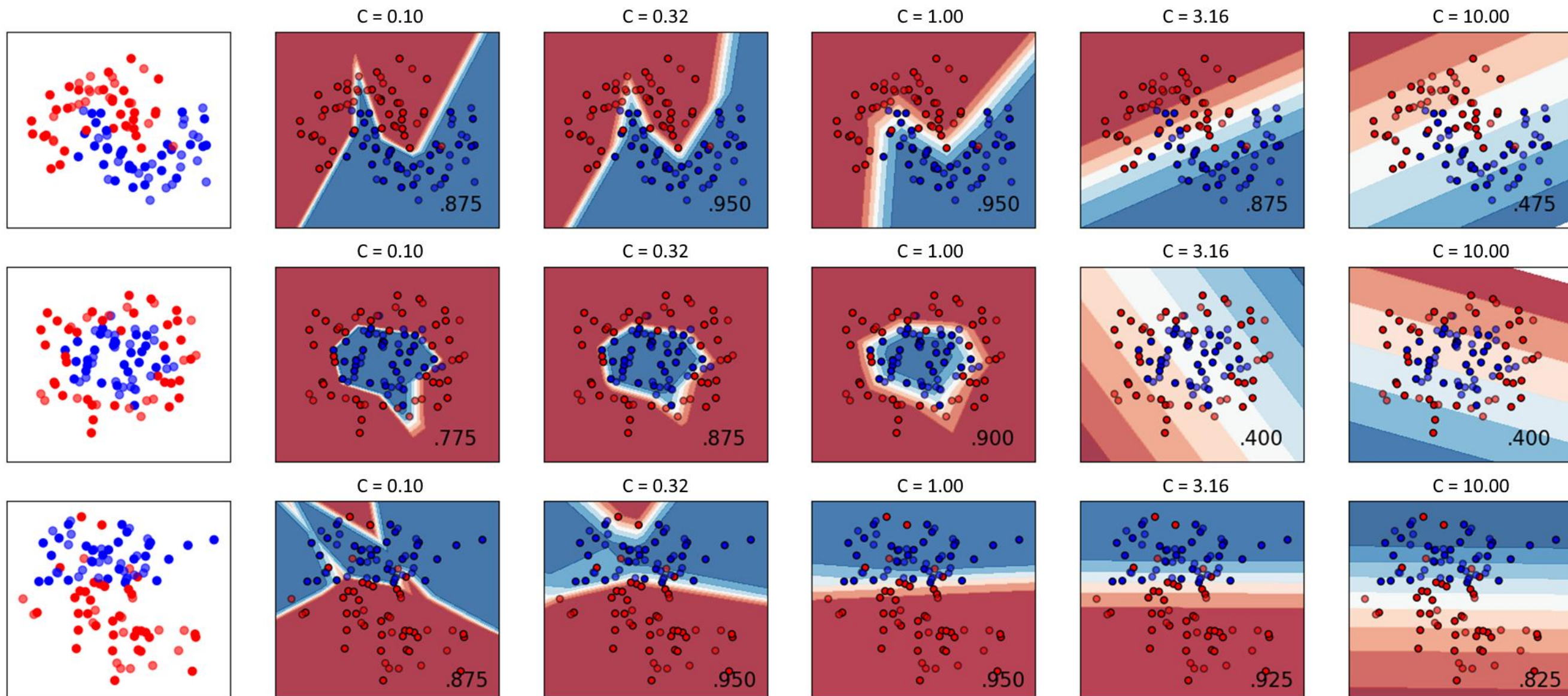
- Al igual que un Perceptron, podemos definir fácilmente una función de pérdida:

$$J(W) = \frac{1}{2n} \sum_{i=1}^n (f(x_i; W) - y_i)^2$$

- Sin embargo, acá debemos considerar 3 nuevos elementos:
 1. Ahora tenemos muchos más parámetros que antes, ¿cómo puedo controlar la complejidad?

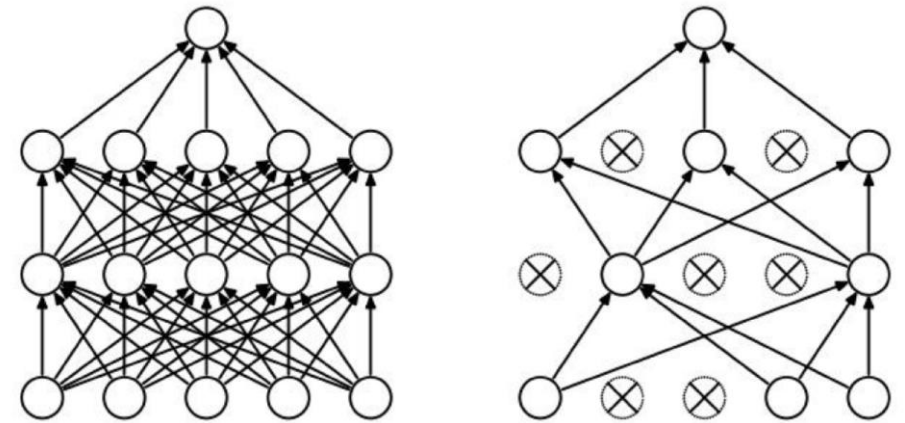
Podemos tomar prestados conceptos de los SVM:

$$J(W) = \frac{C}{2} \|W\|^2 + \frac{1}{2n} \sum_{i=1}^n (f(x_i; W) - y_i)^2$$



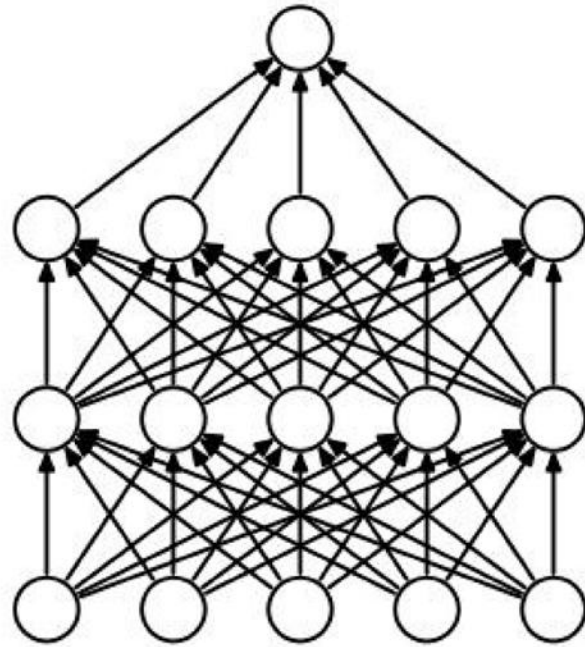
También podemos controlar el sobreajuste “manipulando” la estructura de la red

- Capas densas del final son igualmente propensas a sobreajustarse.
- Para evitar esto, se introdujo en 2012 una técnica llamada **Dropout**, aplicada a las capas densas.
- **Dropout** “apaga” aleatoriamente neuronas en estas capas durante el entrenamiento.
- El efecto neto es que las neuronas no pueden “confiar” en las otras, por lo que aprenden representaciones más robustas.

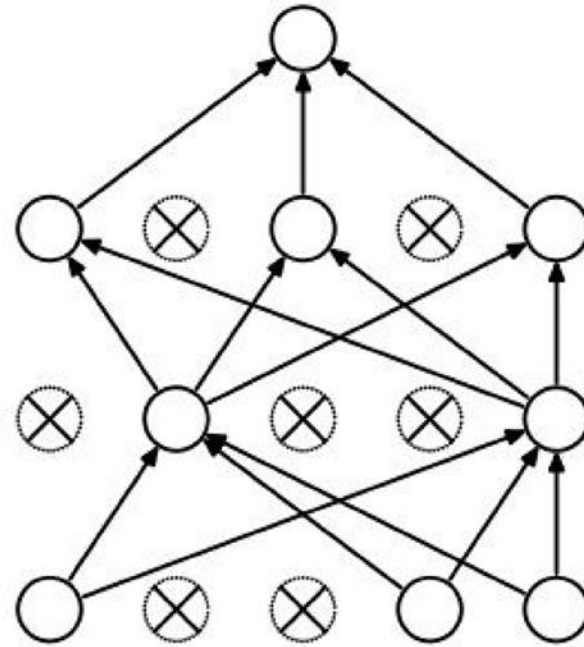


Regularization: **Dropout**

“randomly set some neurons to zero in the forward pass”



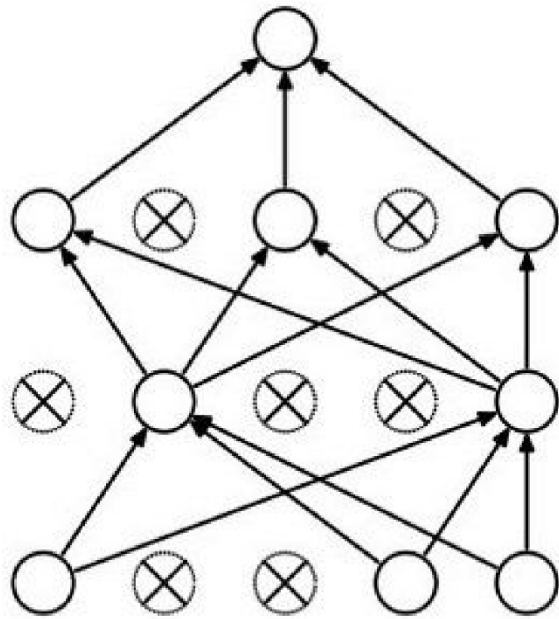
(a) Standard Neural Net



(b) After applying dropout.

Waaaaait a second...

How could this possibly be a good idea?

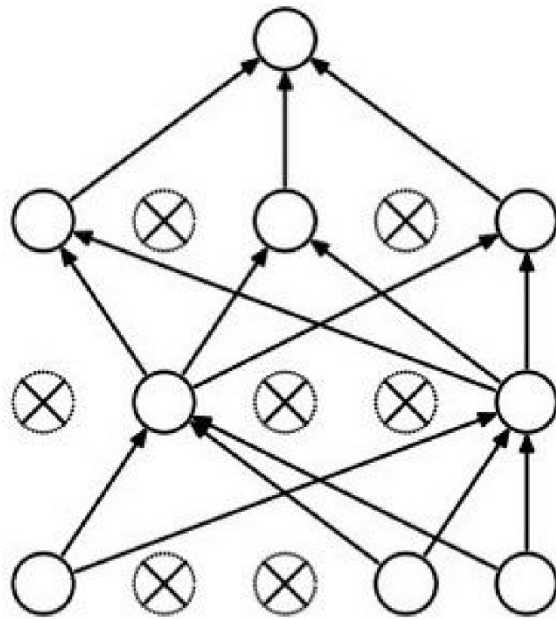


Forces the network to have a redundant representation.



Waaaaait a second...

How could this possibly be a good idea?



Another interpretation:

Dropout is training a large ensemble of models (that share parameters).

Each binary mask is one model, gets trained on only ~one datapoint.

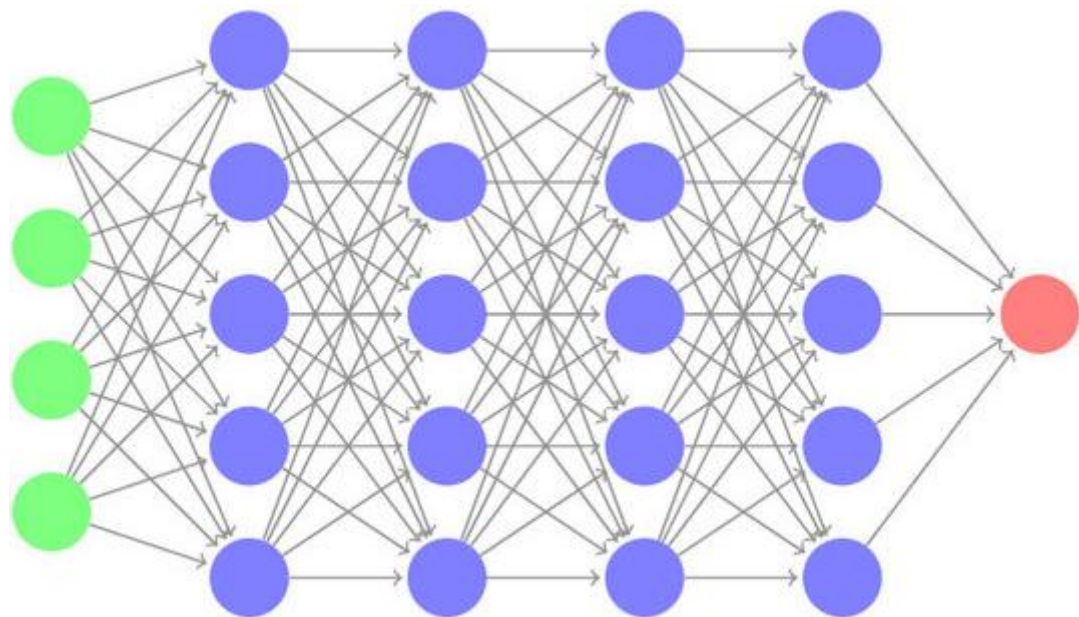
¿Cómo queda entonces el problema de aprendizaje de un MLP?

- Al igual que un Perceptron, podemos definir fácilmente una función de pérdida:

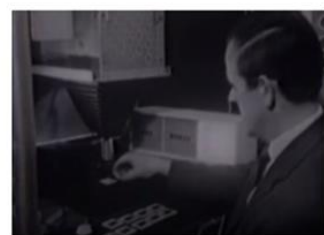
$$J(W) = \frac{1}{2n} \sum_{i=1}^n (f(x_i; W) - y_i)^2$$

- Sin embargo, acá debemos considerar 3 nuevos elementos:
 1. Ahora tenemos muchos más parámetros que antes, ¿cómo puedo controlar la complejidad?
Podemos tomar prestados conceptos de los SVM o usar técnicas como dropout.
 2. ¿Cómo minimizo la función objetivo?

Cálculo de la derivada no es tan directo como con un Perceptron



Al tener composición de funciones, debemos aplicar de manera cuidadosa la regla de la cadena



1958 Perceptron

1974 Backpropagation



Handwritten Recognition

1998



General Object
Detection

2008

No more funds

awkward silence (AI Winter)

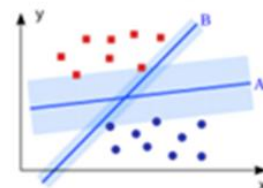
1969

Perceptron criticized



1995

SVM reigns



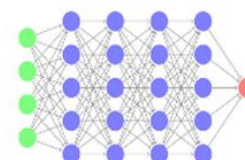
2005

Human
Detection



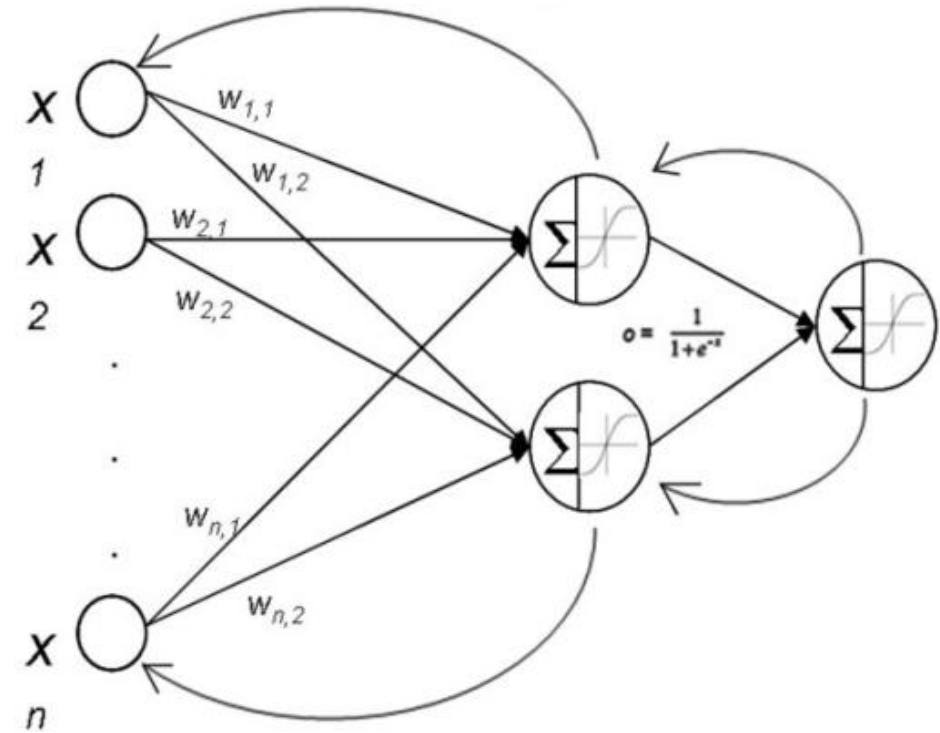
2012

Deep Learning
Rises

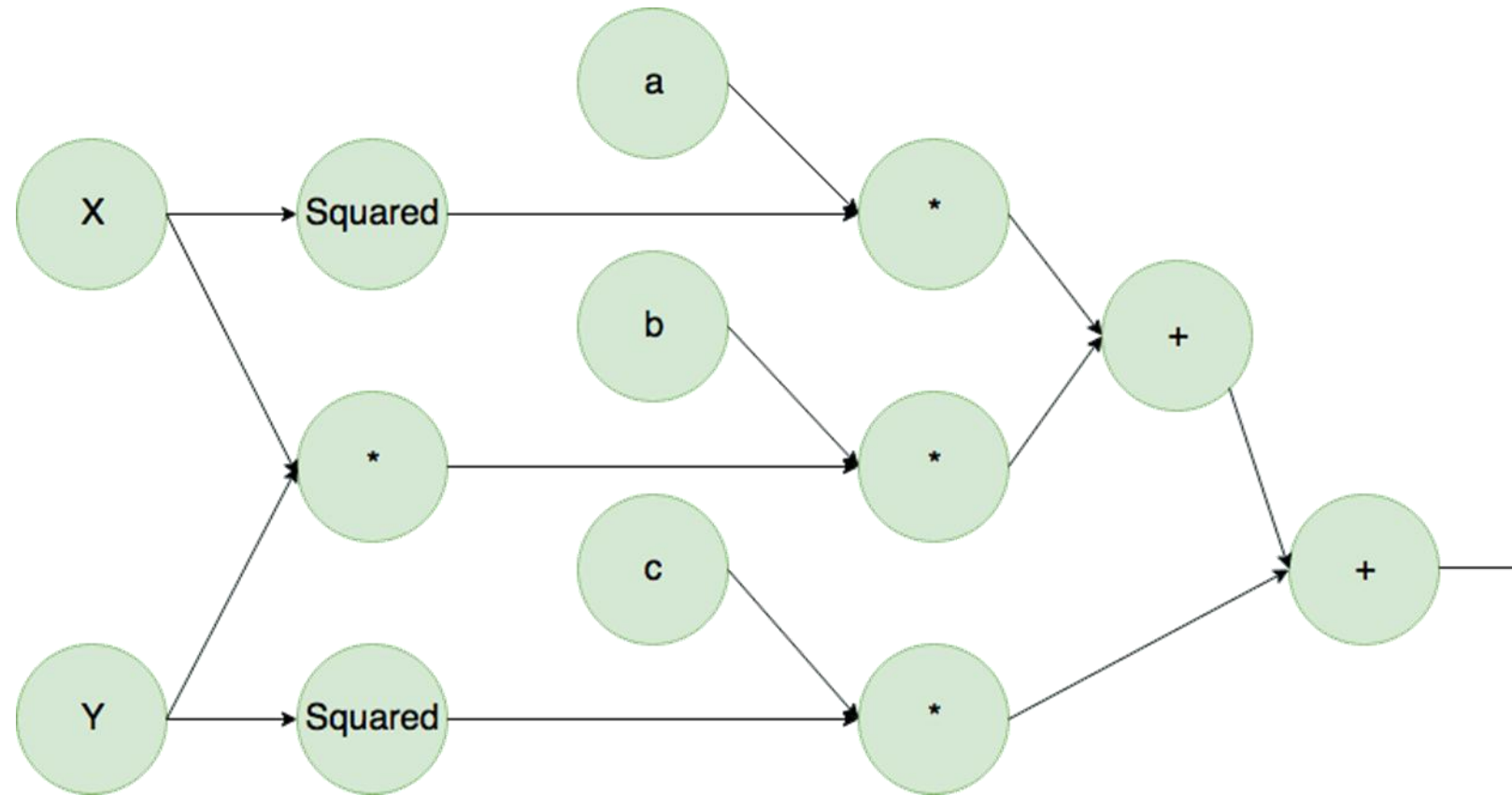


Backpropagation permite entrenar MLPs al **propagar** la *señal* del **error** a todos los nodos

- No es otra cosa que la aplicación recursiva de la **regla de la cadena**.
- En conjunto con el método del descenso del gradiente, permite teóricamente entrenar **redes de profundidad arbitraria**.
- Aplicable a cualquier tipo de red y función diferenciable.

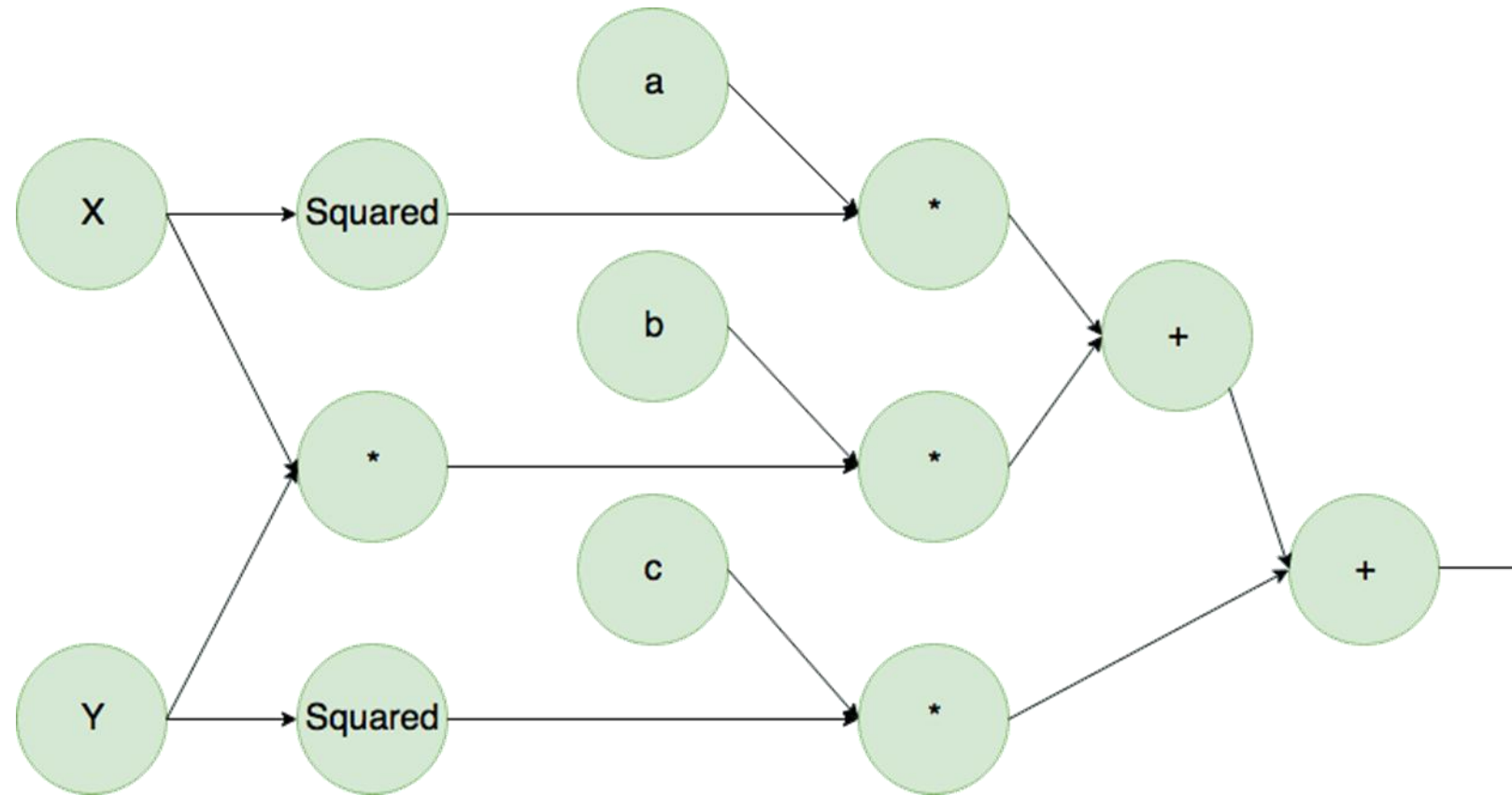


Ahora cambiaremos un poco de perspectiva sobre las redes, y comenzaremos a hablar de **programación diferenciable** y **grafos de cómputo**

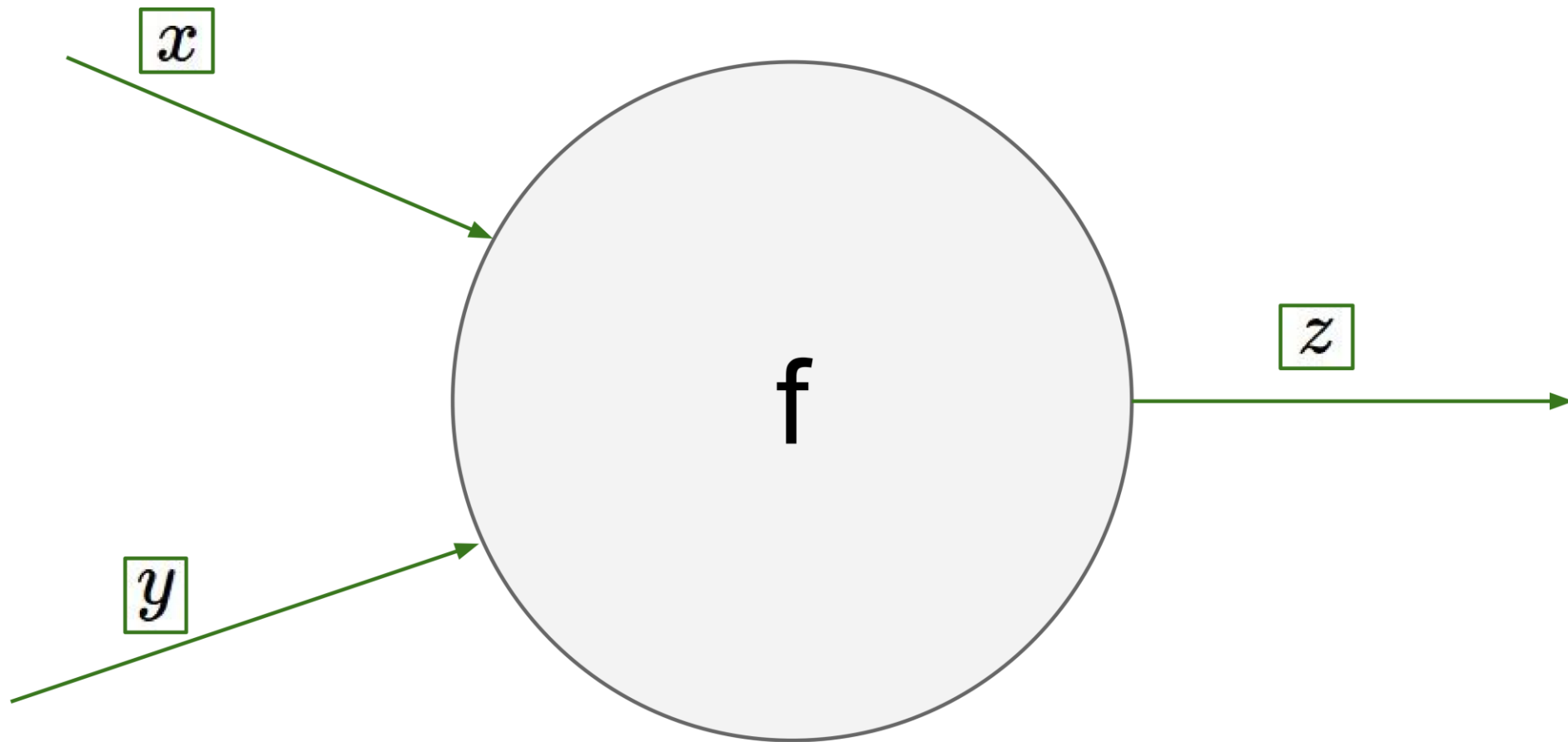


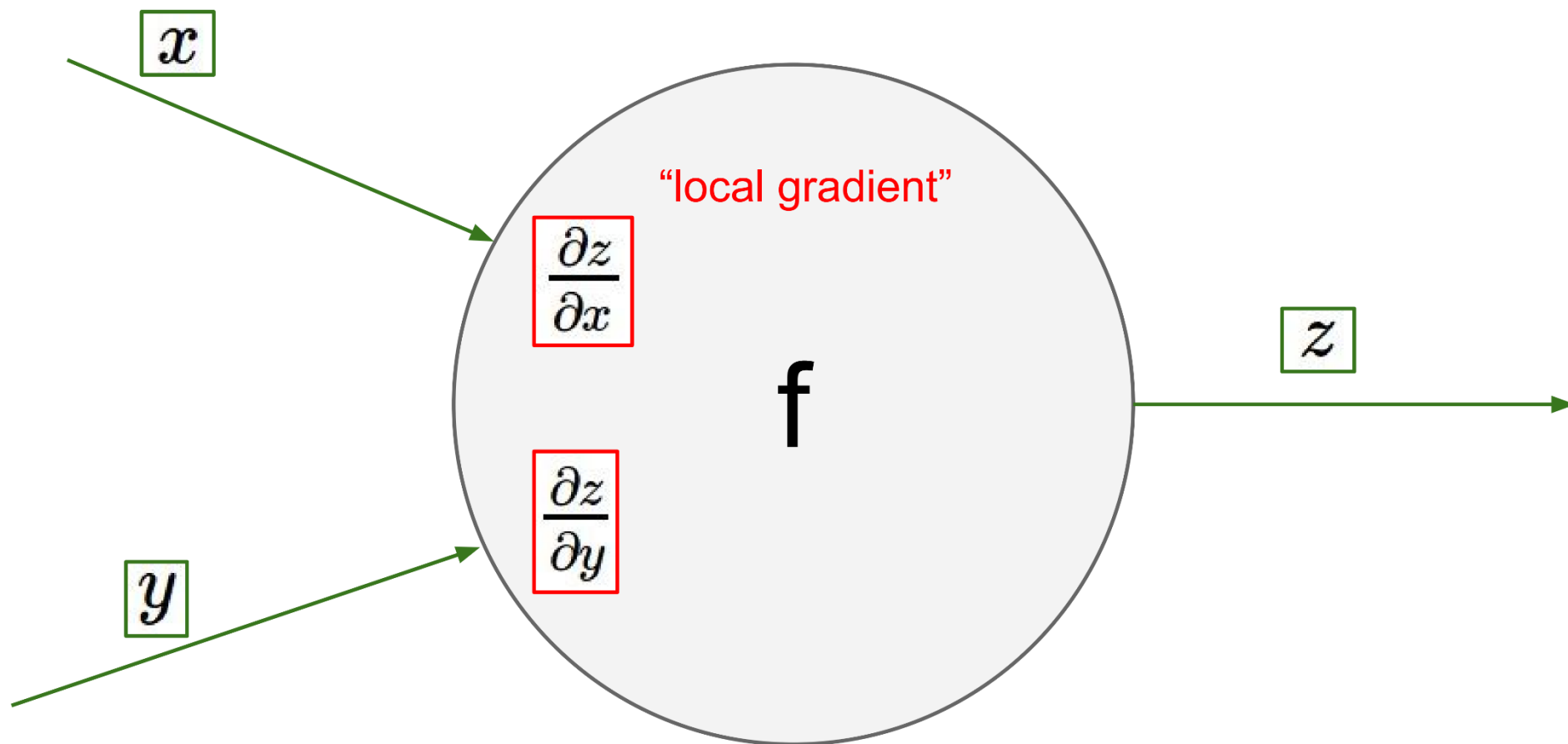
¿Qué función representa este grafo?

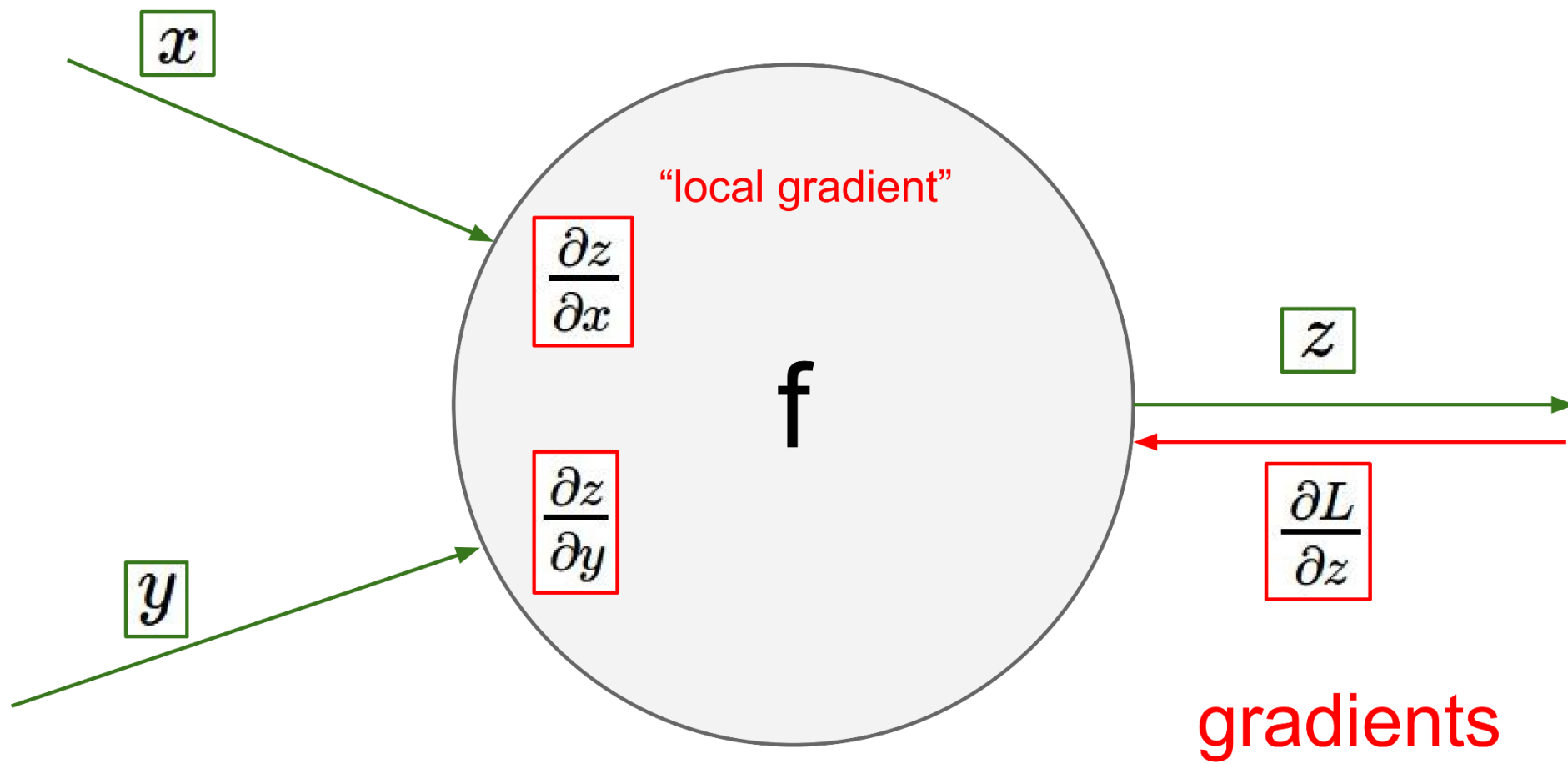
Un grafo de cómputo representa **todas las operaciones** realizadas en una función para obtener su salida

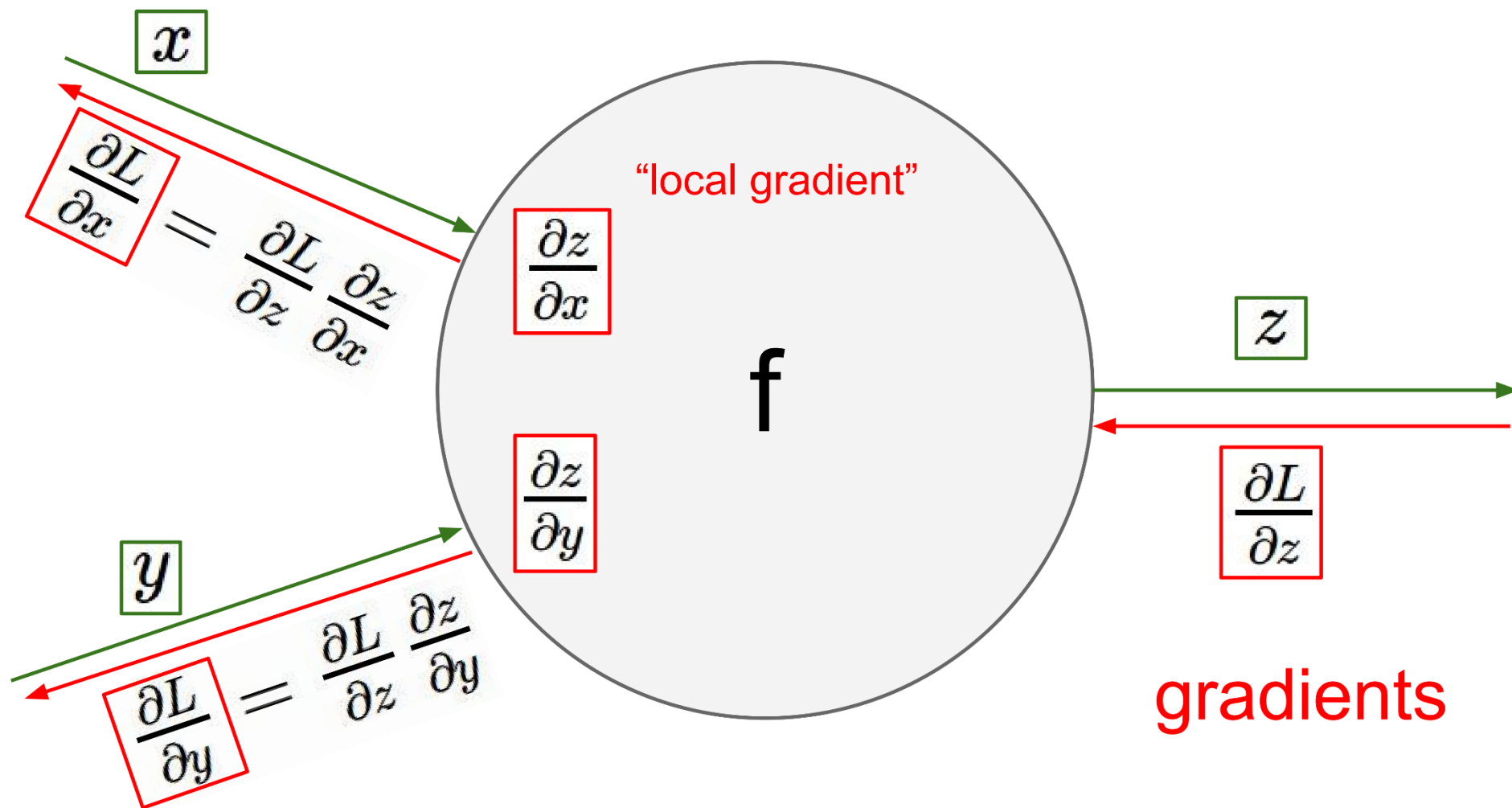


¿Y qué relación tiene esto, por ejemplo, con un MLP?

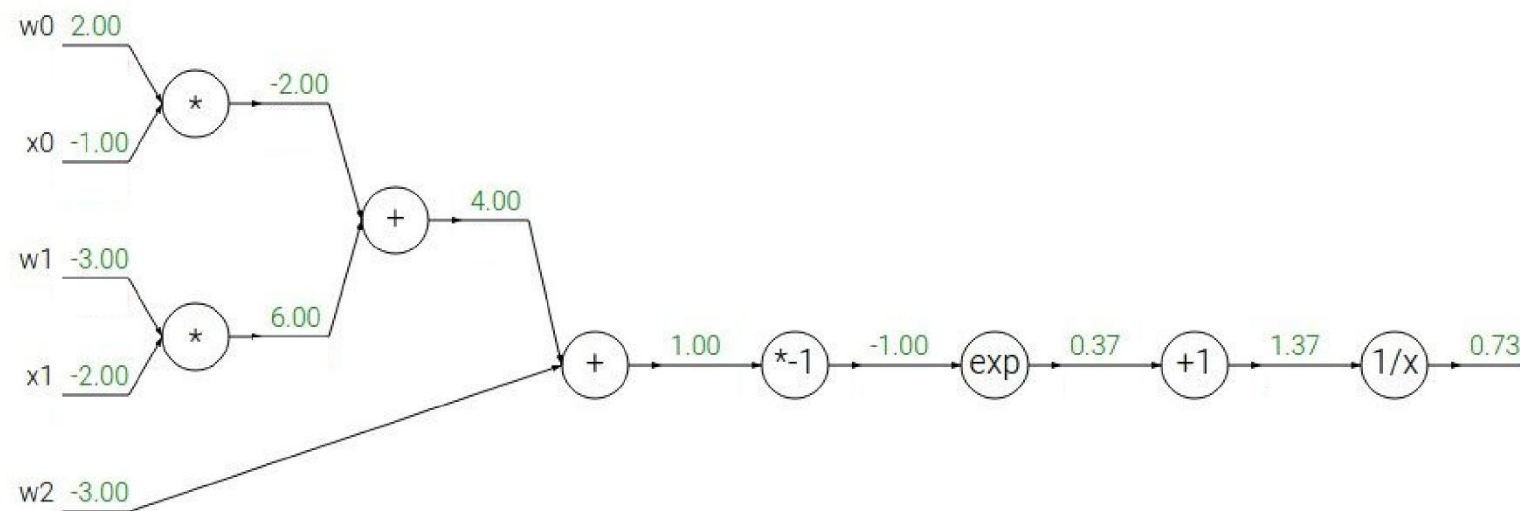




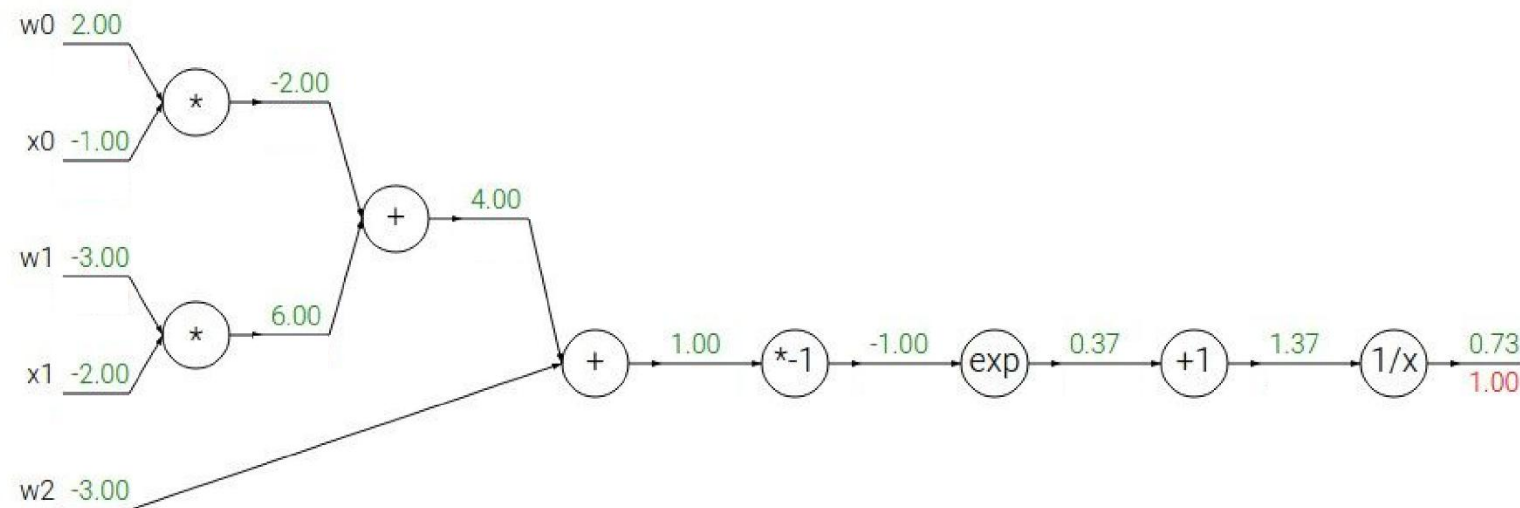




Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$

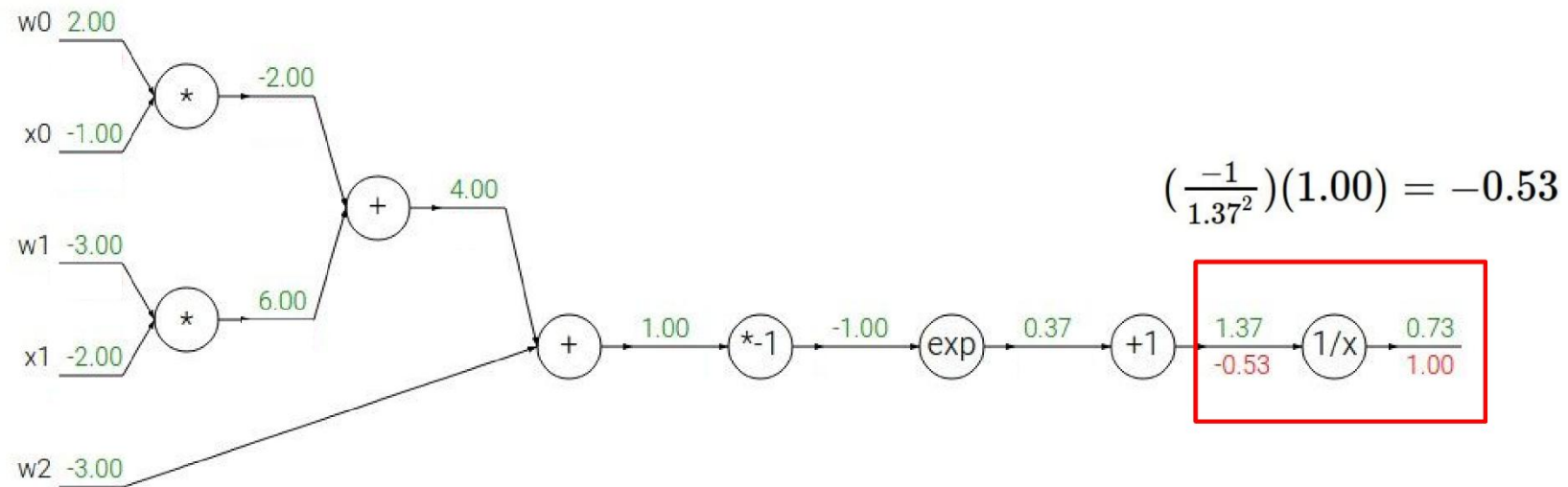


Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



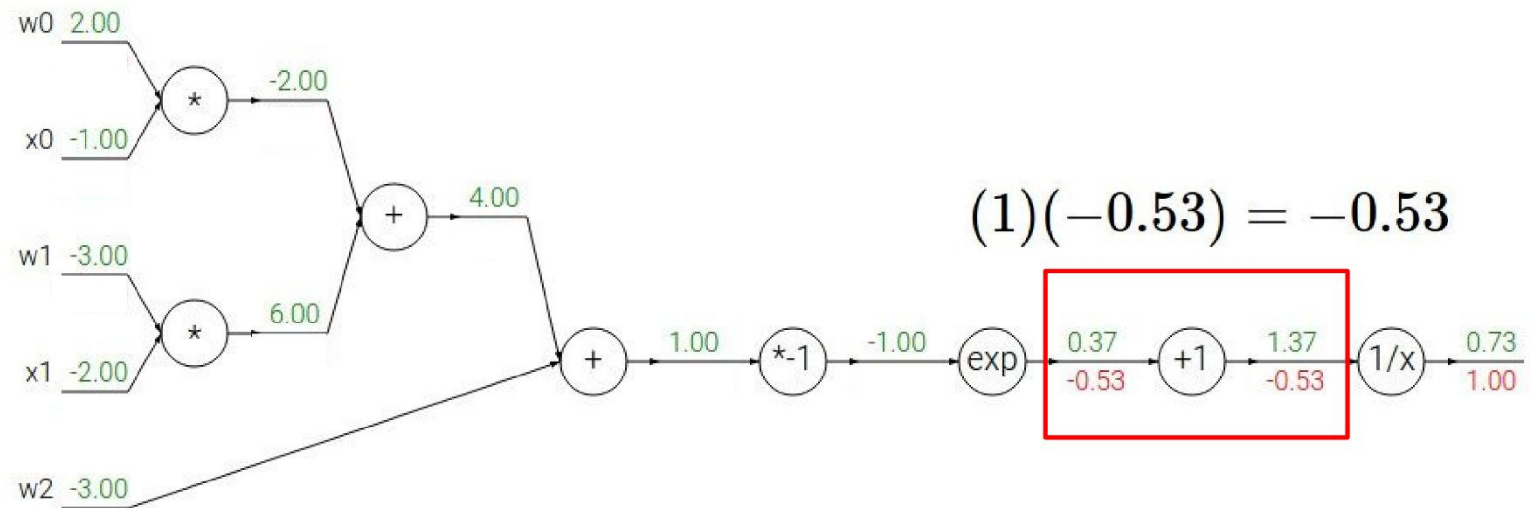
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

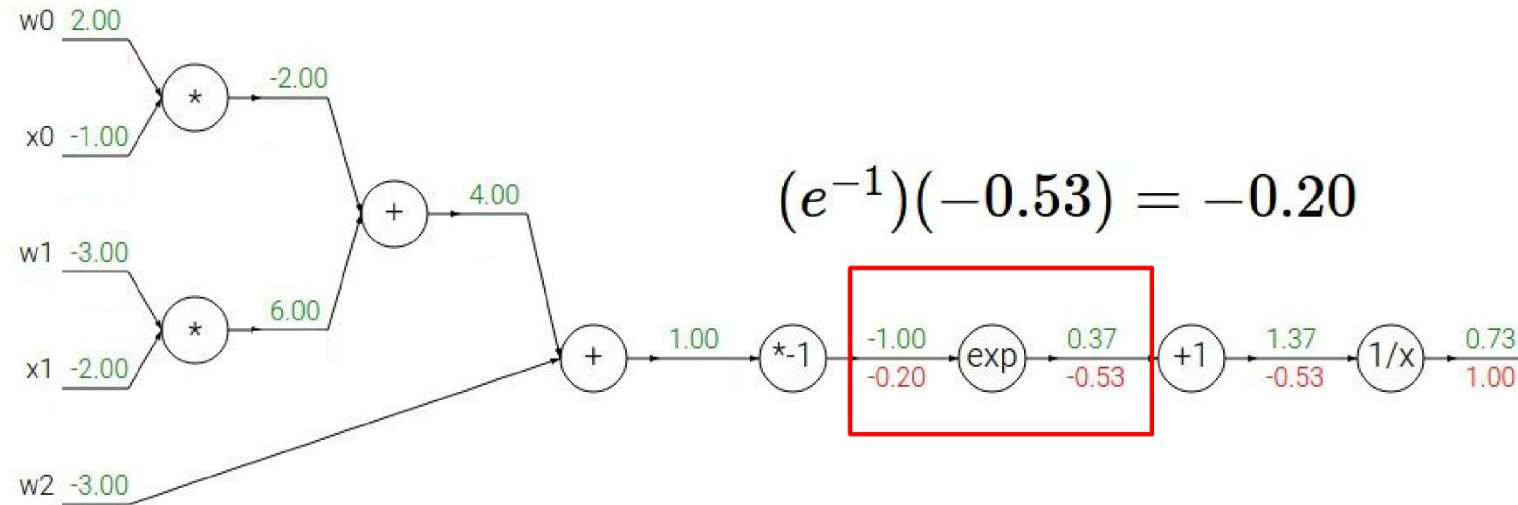
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$$(1)(-0.53) = -0.53$$

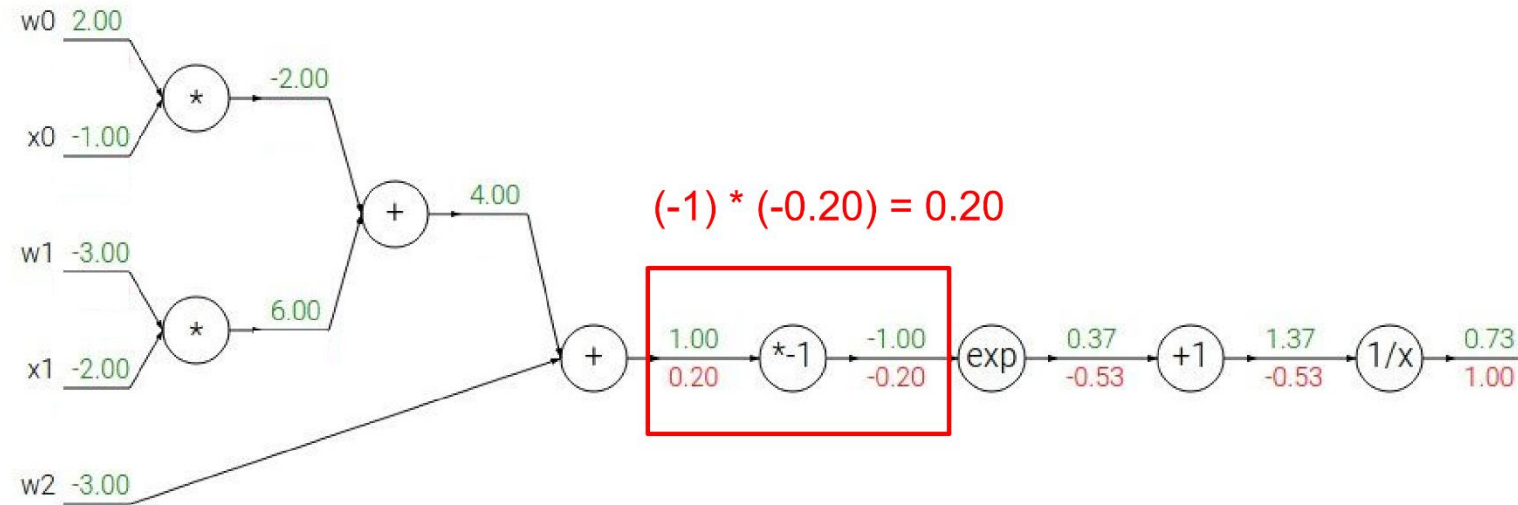
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



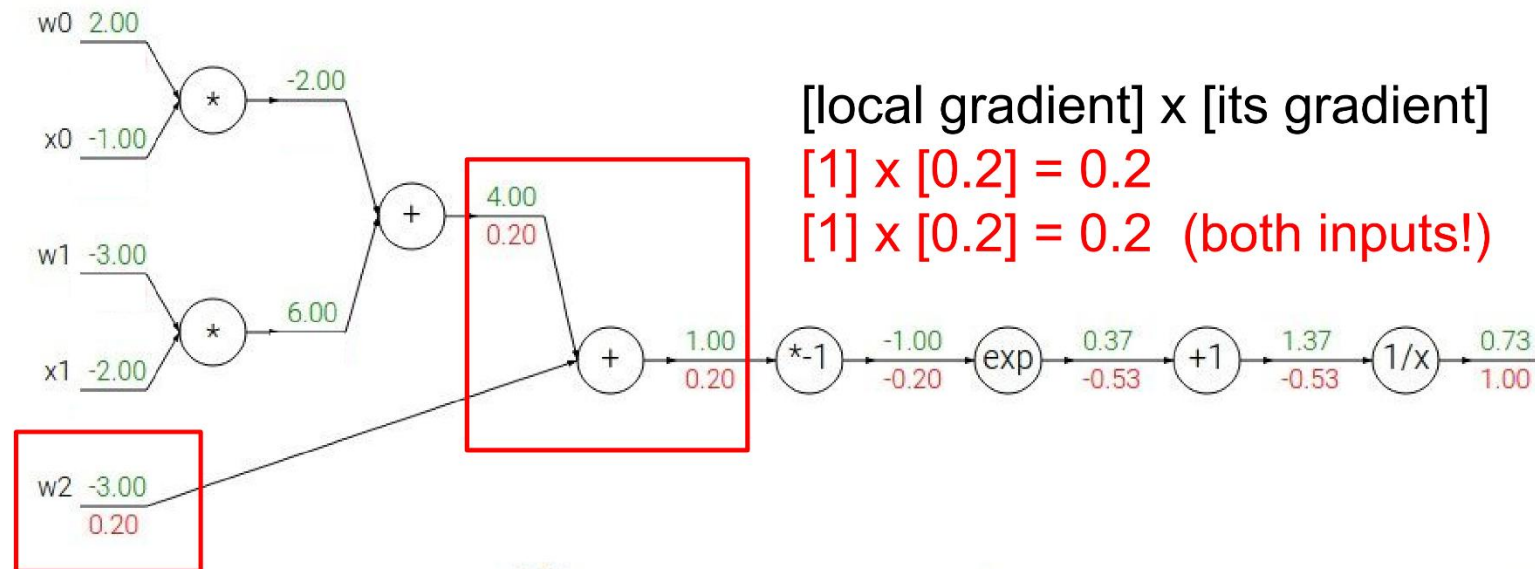
$f(x) = e^x \quad \rightarrow \quad \frac{df}{dx} = e^x$		$f(x) = \frac{1}{x} \quad \rightarrow \quad \frac{df}{dx} = -1/x^2$
$f_a(x) = ax \quad \rightarrow \quad \frac{df}{dx} = a$		$f_c(x) = c + x \quad \rightarrow \quad \frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



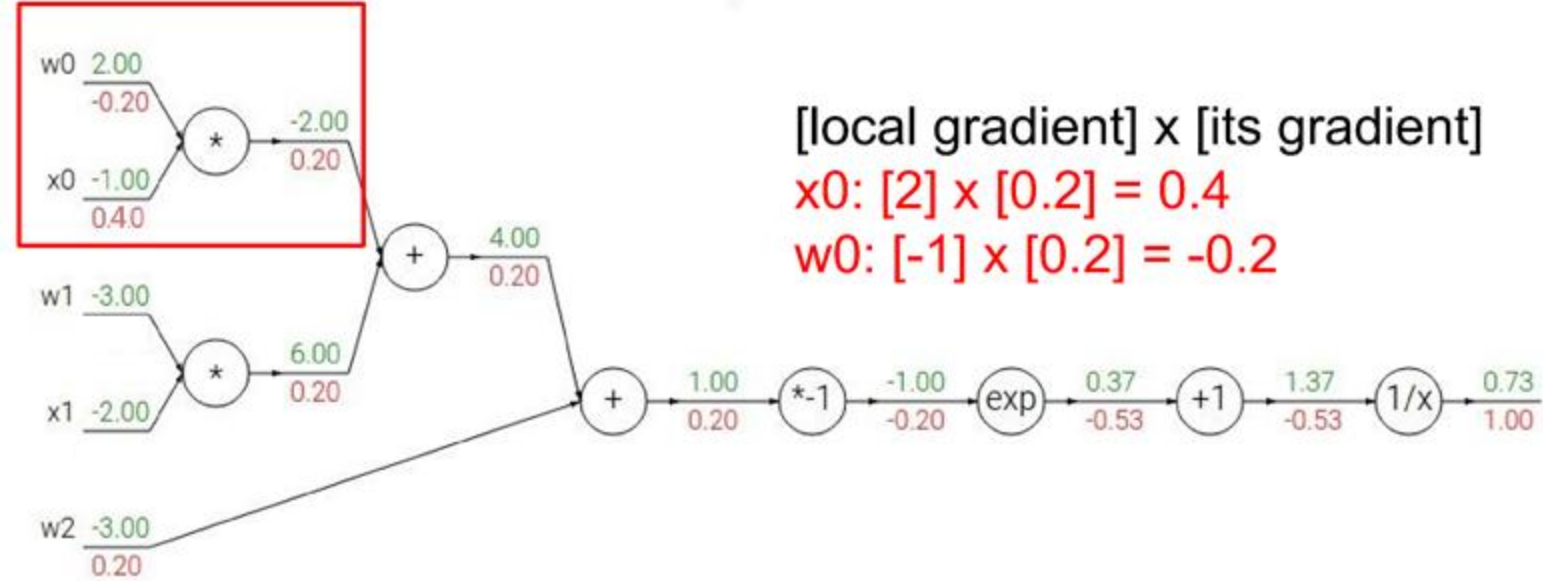
$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

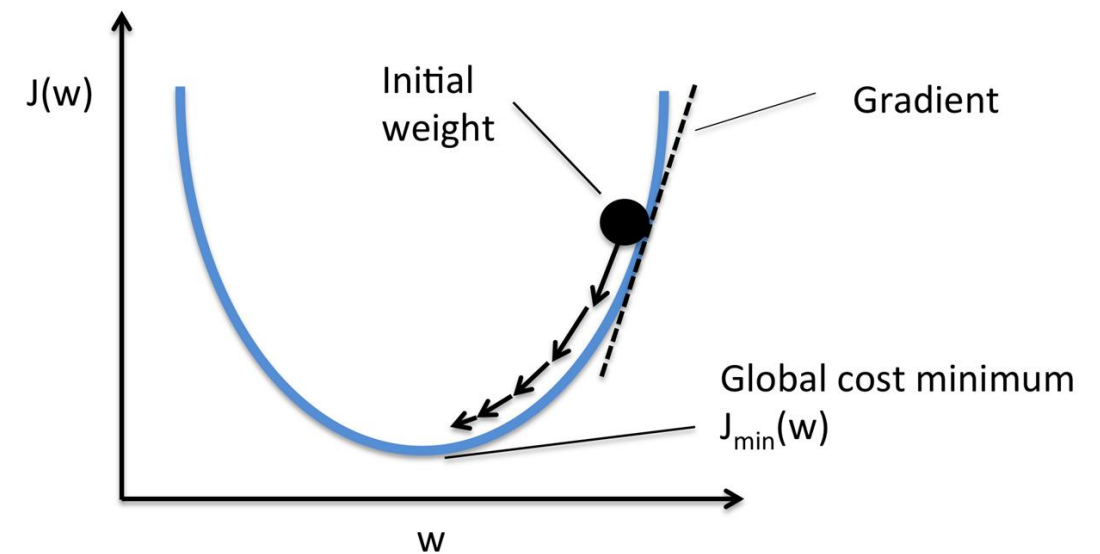
Another example: $f(w, x) = \frac{1}{1 + e^{-(w_0 x_0 + w_1 x_1 + w_2)}}$



$f(x) = e^x$	$\rightarrow$	$\frac{df}{dx} = e^x$		$f(x) = \frac{1}{x}$	$\rightarrow$	$\frac{df}{dx} = -1/x^2$
$f_a(x) = ax$	$\rightarrow$	$\frac{df}{dx} = a$		$f_c(x) = c + x$	$\rightarrow$	$\frac{df}{dx} = 1$

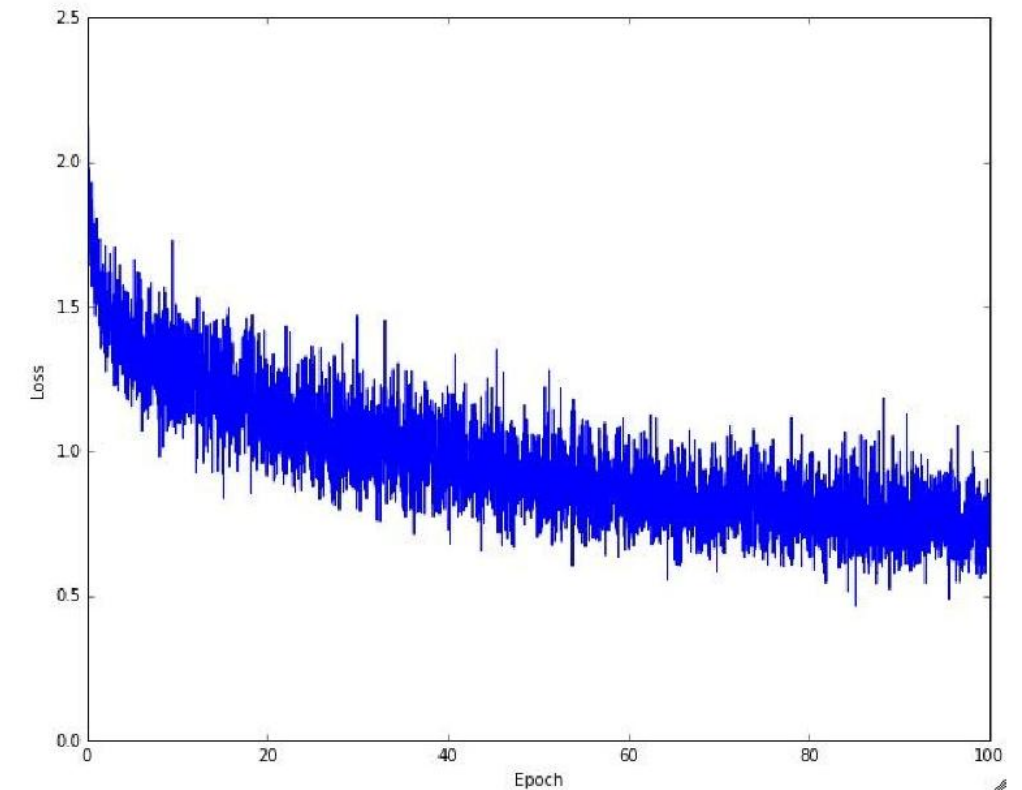
Descenso de gradiente puede ser demasiado demandante computacionalmente

- Para redes y/o o conjuntos de datos grandes, el cálculo de la función y su derivada toman un tiempo considerable.
- Si bien esto puede acelerarse utilizando GPUs, este tipo de escalamiento es costoso.
- ¿Existe algún supuesto que podamos utilizar sobre los datos que pudiera ayudarnos?

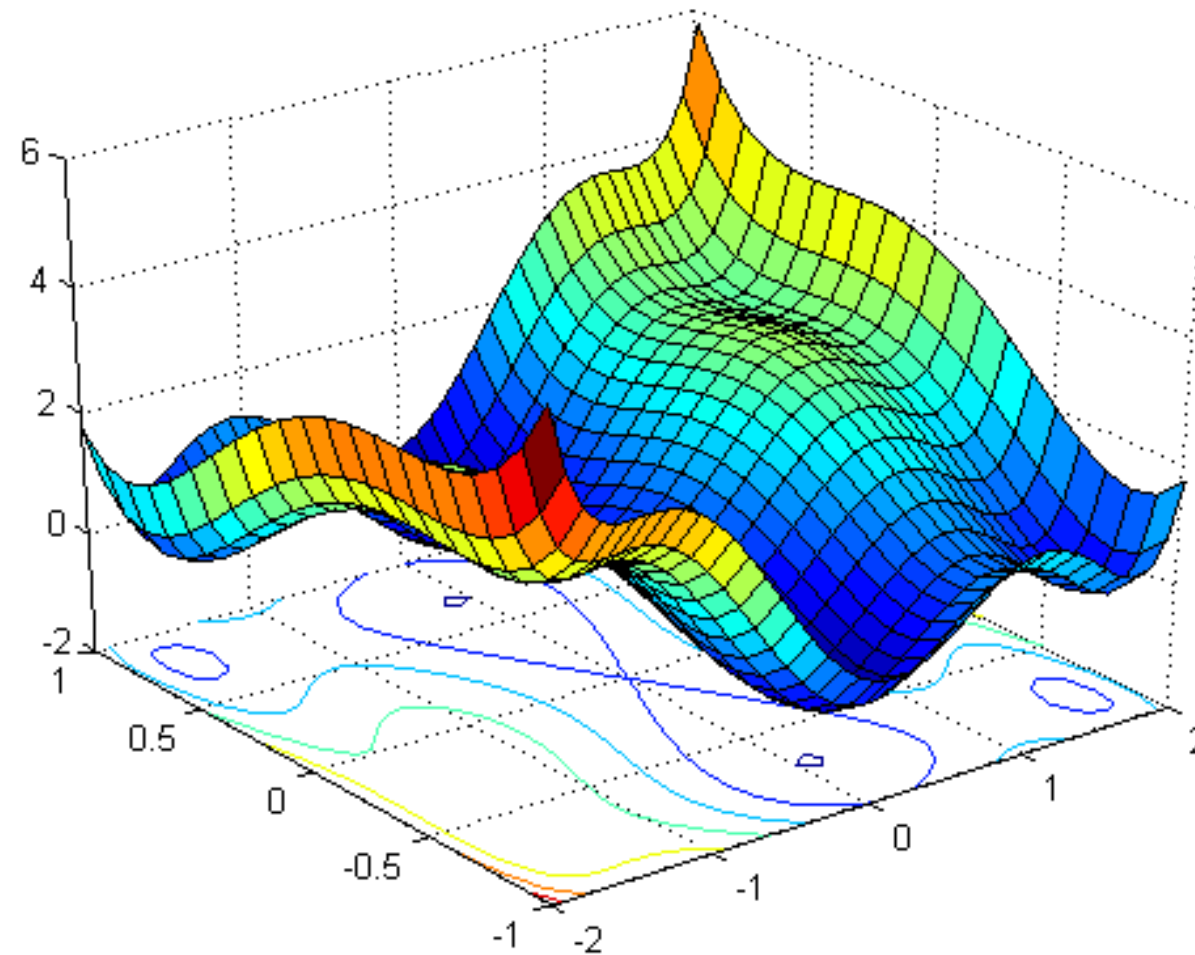


Stochastic Gradient Descent (SGD) aprovecha la fuerza de los promedios

- Consiste en usar un mini-batch de ejemplos para calcular el gradiente para cada descenso, en vez de todos los ejemplos.
- Esta aproximación es un estimador insesgado de la esperanza del gradiente real.
- Existen garantías teóricas de convergencia (tomando un learning rate suficientemente pequeño)
- Tamaños comunes para un mini-batch son 32/64/128/256
- Estocasticidad permite incluso evitar algunos puntos críticos de la función, lo que nos lleva el siguiente punto...

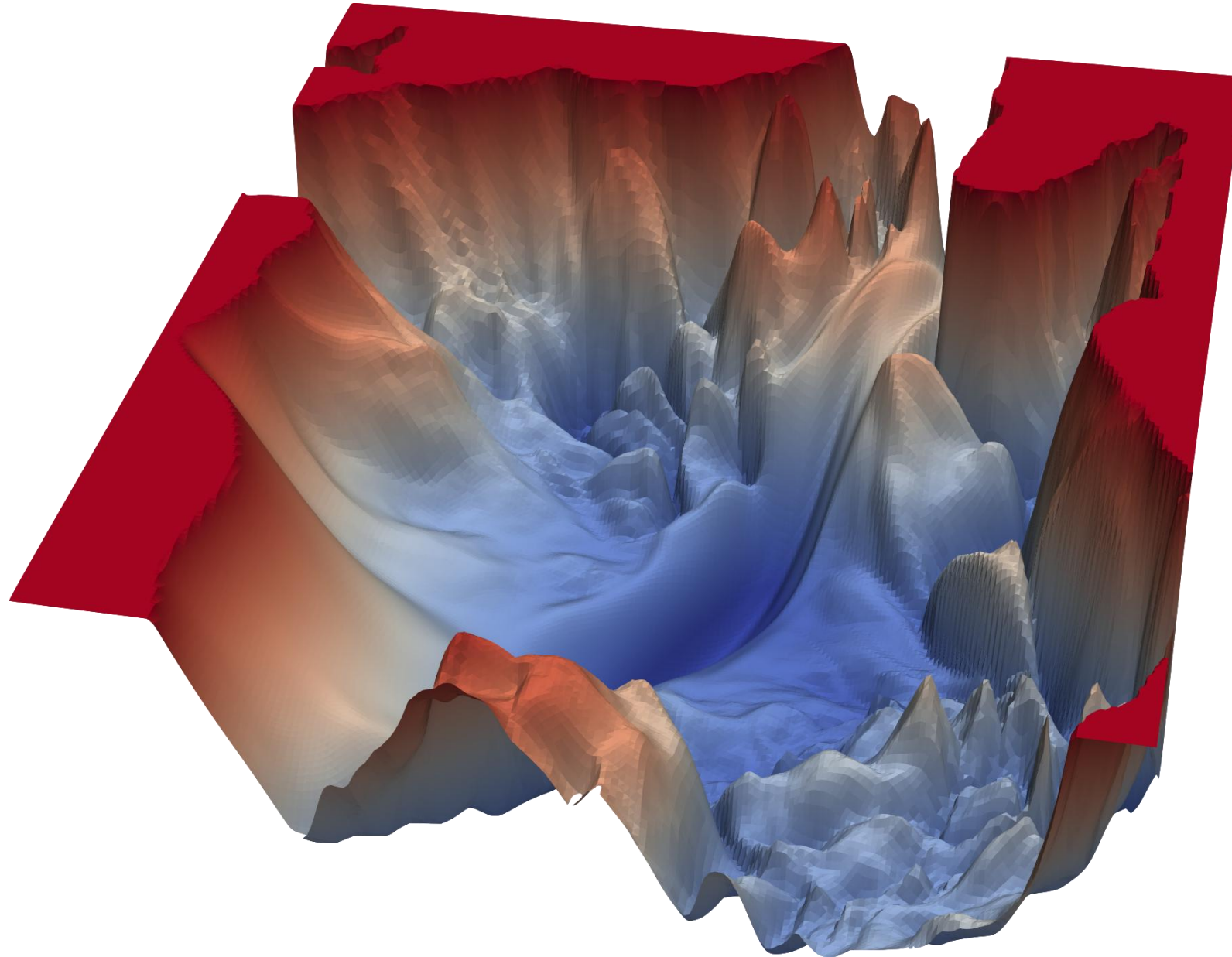


Debido a la estructura de las redes, la función objetivo es no convexa



Pero en realidad ni siquiera es así en la práctica

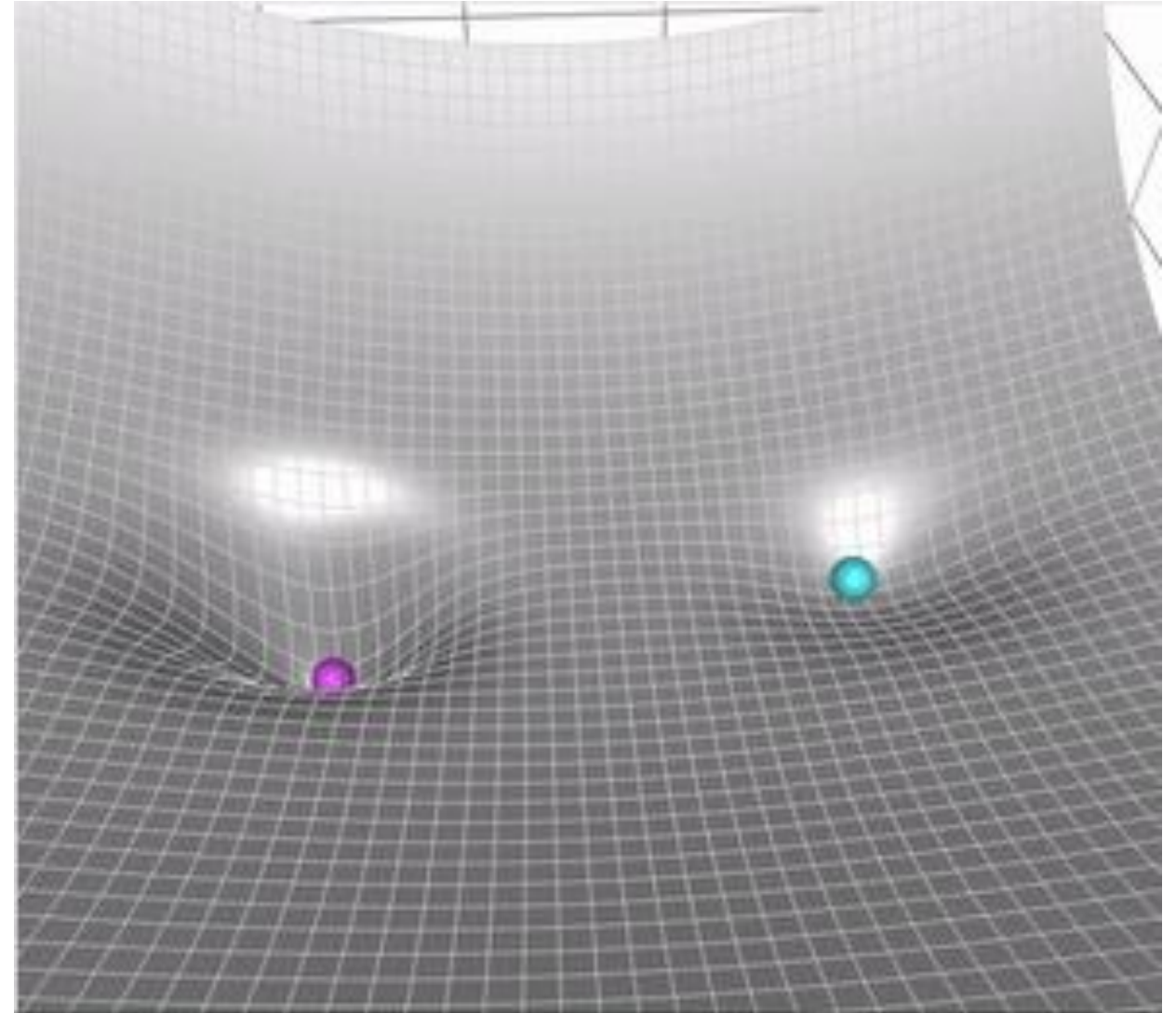
La realidad es mucho peor (en otras palabras, el descenso de gradiente estocástico no sirve)



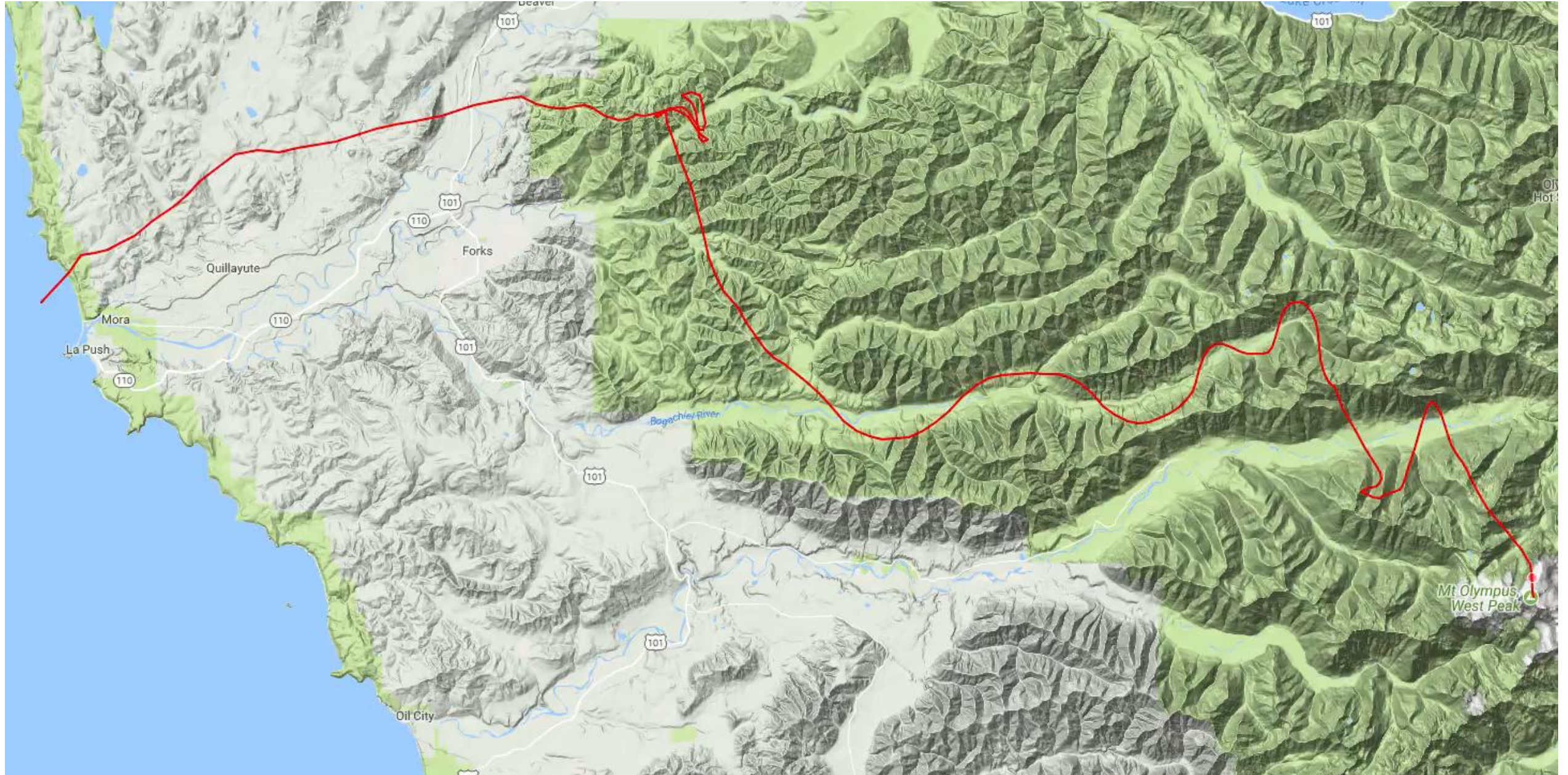
Momentum permite aminorar riesgo de quedar pegado en puntos críticos

$$v_t = \gamma v_{t-1} + \eta \nabla_{\theta} J(\theta_t)$$

$$\theta_{t+1} = \theta_t - v_t$$



Un ejemplo muy “bonito” puede construirse utilizando datos geográficos de elevación



¹<https://fosterelli.co/executing-gradient-descent-on-the-earth>

¿Cómo queda entonces el problema de aprendizaje de un MLP?

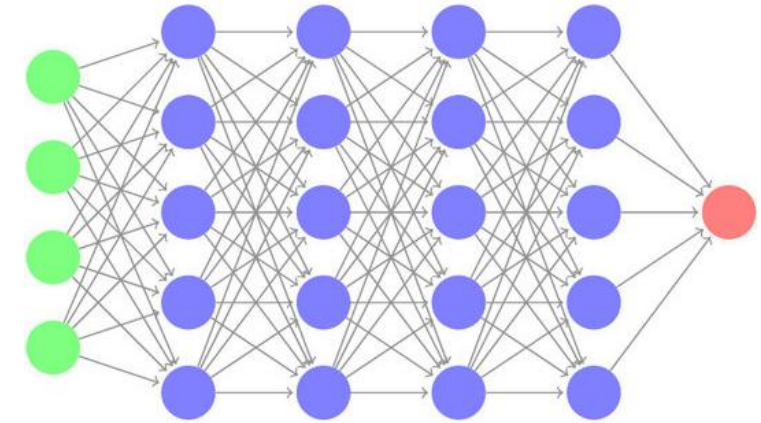
- Al igual que un Perceptron, podemos definir fácilmente una función de pérdida:

$$J(W) = \frac{1}{2n} \sum_{i=1}^n (f(x_i; W) - y_i)^2$$

- Sin embargo, acá debemos considerar 3 nuevos elementos:
 1. Ahora tenemos muchos más parámetros que antes, ¿cómo puedo controlar la complejidad?
Podemos tomar prestados conceptos de los SVM o usar técnicas como dropout.
 2. ¿Cómo minimizo la función objetivo?
Backpropagation sobre un grafo de cómputo + SGD + momentum
 3. ¿Qué límites tiene un MLP en cuanto a lo que puede aprender?
Spoiler: no tiene límites

¿Hasta dónde podemos llegar con un MLP?

- De hecho, hasta donde uno quiera.
- De acuerdo al “*teorema de aproximación universal*”, un MLP con 1 capa oculta puede aproximar cualquier función continua, con precisión arbitraria.
- Además, un MLP con 2 capas ocultas es capaz de aproximar cualquier función, con precisión arbitraria.
- Una buena “mostración” del teorema puede verse en <https://www.youtube.com/watch?v=ljqkc7OLenI>



Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2613 - Inteligencia Artificial

Entrenamiento de Redes Neuronales

Hans Löbel

Dpto. Ingeniería de Transporte y Logística
Dpto. Ciencia de la Computación